

Notas de estudio — Clase 1

Fundamentos de Teoría de Códigos y Códigos Reed–Solomon

Manuel Puebla

Seminario CCSD-UBA-LambdaClass

Resumen

Notas personales para preparar la primera de cuatro clases del seminario sobre Teoría de Códigos, Low-Degree Testing y PIOPs. El objetivo es construir, desde lo más elemental, las nociones de código, distancia, código lineal y código MDS, hasta poder demostrar que los códigos de Reed–Solomon saturan la cota de Singleton. Se incluye un apéndice sobre la interacción entre el Teorema Fundamental del Álgebra (en su forma útil aquí: número de raíces vs. grado) y los parámetros (n, k, d, R) de un código basado en evaluación de polinomios. Las referencias principales son [1], [2], [4] y [5].

Índice

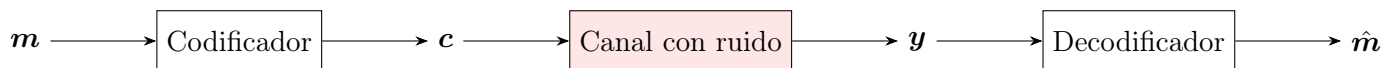
1. Motivación: ¿por qué códigos?	1
2. El objeto “código”	6
2.1. Definiciones básicas	6
2.2. Dos ejemplos de juguete que ya enseñan todo	7
3. La métrica de Hamming	7
3.1. Repaso mínimo: espacios métricos	7
3.2. Distancia de Hamming	8
3.3. Distancia mínima del código	8
3.4. Bola de Hamming, detección y corrección	9
3.5. Decodificación por lista	10
3.6. Modelos de error: Hamming vs. Shannon	15
4. De códigos cualesquiera a códigos lineales	15
4.1. Por qué pedir estructura	15
4.2. Repaso mínimo: cuerpos finitos	15
4.3. Códigos lineales	16
4.4. La identidad clave: $d = \text{mín peso}$	16
4.5. Matriz generadora y matriz de paridad	16
4.6. Decodificación por síndrome	17
4.7. El ejemplo dorado: el código de Hamming $[7, 4, 3]_2$	17
5. Códigos de Reed–Solomon	18
5.1. El instrumento: polinomios sobre un cuerpo	18
5.2. Construcción del código	18
5.3. El teorema central	19
5.4. Una matriz generadora explícita	19
5.5. Un ejemplo numérico completo	20
5.6. Reed–Solomon como “el mejor código posible”	21

A. Apéndice: TFA, palabras de código y los parámetros (n, k, d, R)	21
A.1. El esqueleto algebraico	21
A.2. Cuatro consecuencias en cadena	22
A.3. Tabla de casos numéricos: RS sobre $\mathbb{F}_q$ con q grande	22
A.4. Una analogía aritmética	23
B. Síntesis y puente a la Clase 2	23

1. Motivación: ¿por qué códigos?

El problema clásico (Shannon, 1948)

Pensemos en la situación más concreta posible. Alice quiere enviarle a Bob una secuencia de bits. Entre ellos hay un canal ruidoso: con cierta probabilidad cada bit se invierte (un *flip*). Bob recibe una secuencia que se parece, pero no necesariamente coincide, con la que envió Alice.



¿Cómo agrega Alice *redundancia* a su mensaje para que Bob, viendo $\mathbf{y}$, pueda recuperar $\mathbf{m}$ con probabilidad alta? Esa es la pregunta que Claude Shannon formalizó en su artículo seminal de 1948 [8], fundando simultáneamente la *Teoría de la Información* y la *Teoría de Códigos*.

El canal como modelo probabilístico. En la visión de Shannon, los errores no son adversariales: son una consecuencia inevitable de la naturaleza física del canal. Cada coordenada del codeword es perturbada *aleatoriamente*. La formalización es un objeto matemático bien definido.

Definición 1.1 (Canal discreto sin memoria). Un *canal discreto sin memoria* (DMC) consta de un alfabeto de entrada $\mathcal{X}$, un alfabeto de salida $\mathcal{Y}$, ambos finitos, y una colección de probabilidades de transición

$$W(y | x) = \Pr[Y = y | X = x], \quad x \in \mathcal{X}, y \in \mathcal{Y}.$$

La condición *sin memoria* significa que al transmitir una secuencia $\mathbf{x} = (x_1, \dots, x_n)$, la palabra recibida $\mathbf{y} = (y_1, \dots, y_n)$ se distribuye según

$$\Pr[\mathbf{Y} = \mathbf{y} | \mathbf{X} = \mathbf{x}] = \prod_{i=1}^n W(y_i | x_i).$$

Ejemplo 1.2 (Canal binario simétrico). El *canal binario simétrico* de parámetro $p \in [0, 1/2]$, denotado BSC(p), tiene $\mathcal{X} = \mathcal{Y} = \{0, 1\}$ y

$$W(y | x) = \begin{cases} 1 - p & \text{si } y = x, \\ p & \text{si } y \neq x. \end{cases}$$

Equivalentemente, $Y = X \oplus E$ con $E \sim \text{Bernoulli}(p)$ independiente de X . El parámetro p se llama *probabilidad de flip* o *crossover probability*.

El mensaje $\mathbf{M}$, el codeword transmitido $\mathbf{X} = E(\mathbf{M})$, el patrón de error $\mathbf{E}$ y la palabra recibida $\mathbf{Y} = \mathbf{X} \oplus \mathbf{E}$ son ahora variables aleatorias. La calidad de un esquema codificador-decodificador se mide por su *probabilidad de error*:

$$P_{\text{err}} := \Pr[D(\mathbf{Y}) \neq \mathbf{M}],$$

donde el promedio se toma sobre las distribuciones de $\mathbf{M}$ y del canal. *Promedio*, no peor caso: éste es el rasgo distintivo del marco de Shannon.

Los objetos matemáticos: entropía, información mutua, capacidad. Para responder “¿cuán bueno puede ser un esquema?” Shannon introdujo medidas cuantitativas del azar.

Definición 1.3 (Entropía de Shannon). Sea X una variable aleatoria discreta con valores en $\mathcal{X}$ y distribución $p_X(x) = \Pr[X = x]$. La *entropía* de X es

$$H(X) := - \sum_{x \in \mathcal{X}} p_X(x) \log_2 p_X(x),$$

con convención $0 \log 0 := 0$. Sus unidades son *bits*.

Intuitivamente, $H(X)$ mide la cantidad media de bits necesaria para describir el valor de X . Una variable determinística tiene entropía 0; una uniforme sobre un soporte de tamaño M tiene entropía $\log_2 M$, la máxima posible para ese soporte.

Ejemplo 1.4 (Entropía binaria). Para $X \sim \text{Bernoulli}(p)$,

$$H_2(p) := H(X) = -p \log_2 p - (1-p) \log_2 (1-p), \quad p \in (0, 1).$$

La función H_2 es estrictamente cóncava, se anula en 0 y en 1, y alcanza su máximo $H_2(1/2) = 1$ en $p = 1/2$.

Definición 1.5 (Entropía conjunta y condicional). Para variables aleatorias X, Y con distribución conjunta p_{XY} definimos

$$H(X, Y) := - \sum_{x, y} p_{XY}(x, y) \log_2 p_{XY}(x, y), \quad H(Y | X) := - \sum_{x, y} p_{XY}(x, y) \log_2 p_{Y|X}(y | x).$$

Vale la *regla de la cadena*: $H(X, Y) = H(X) + H(Y | X)$.

Definición 1.6 (Información mutua). La *información mutua* entre X y Y es

$$I(X; Y) := H(Y) - H(Y | X) = H(X) - H(X | Y) = \sum_{x, y} p_{XY}(x, y) \log_2 \frac{p_{XY}(x, y)}{p_X(x) p_Y(y)}.$$

Mide la cantidad media de bits sobre X que “se observan” al medir Y .

Observación 1.7. $I(X; Y) \geq 0$, con igualdad si y solo si X e Y son independientes. Además $I(X; Y) = I(Y; X)$ y $I(X; Y) \leq \min\{H(X), H(Y)\}$.

Definición 1.8 (Capacidad del canal). La *capacidad* de un DMC W es

$$C(W) := \sup_{p_X} I(X; Y),$$

donde el supremo recorre las distribuciones p_X sobre $\mathcal{X}$ y la distribución de Y se obtiene aplicando W a X .

Ejemplo 1.9 (Capacidad del BSC). Para $\text{BSC}(p)$,

$$C(\text{BSC}(p)) = 1 - H_2(p),$$

y el supremo se alcanza con X uniforme en $\{0, 1\}$. En particular: para $p = 0$ (canal limpio) la capacidad es 1 bit por uso del canal; para $p = 1/2$ (ruido total) la capacidad es 0; en el medio, C es estrictamente decreciente en $p \in [0, 1/2]$.

El teorema fundamental. Shannon mostró que esta cantidad —definida abstractamente como un supremo de información mutua— coincide exactamente con la máxima tasa a la que se puede comunicar de forma confiable.

Teorema 1.10 (Codificación con ruido — Shannon 1948). *Sea W un DMC con capacidad $C = C(W)$.*

- (I) (Realizabilidad.) *Para todo $R < C$ y todo $\varepsilon > 0$, existe $n_0 \in \mathbb{N}$ tal que para todo $n \geq n_0$ hay un código $\mathcal{C} \subseteq \mathcal{X}^n$ con $|\mathcal{C}| \geq 2^{nR}$ codewords y una función de decodificación $D : \mathcal{Y}^n \rightarrow \mathcal{C}$ tales que la probabilidad media de error verifica*

$$\frac{1}{|\mathcal{C}|} \sum_{\mathbf{c} \in \mathcal{C}} \Pr[D(\mathbf{Y}) \neq \mathbf{c} \mid \mathbf{X} = \mathbf{c}] \leq \varepsilon.$$

- (II) (Recíproca.) *Si una familia de códigos $\{\mathcal{C}_n\}_{n \geq 1}$ verifica $|\mathcal{C}_n| \geq 2^{nR}$ con $R > C$, entonces la probabilidad media de error queda acotada por debajo, lejos de cero, cuando $n \rightarrow \infty$.*

La capacidad es por lo tanto el umbral exacto: por debajo se puede comunicar arbitrariamente bien (a costa de bloque más largo); por encima, no, sin importar cuán inteligente sea el codificador. Tres rasgos del teorema conviene destacar:

- (1) Es *asintótico*: los códigos existen para n suficientemente grande. No exhibe nada constructivo.
- (2) Es *average-case*: lo que se controla es la probabilidad media de error sobre la distribución del canal, no el peor caso. Un patrón de ruido extremadamente raro puede aún anular un bloque particular.
- (3) La prueba original usa *códigos aleatorios*: muestra que existe un código que cumple la cota promediando sobre todos los códigos posibles. Es un argumento existencial, no algorítmico.

Aplicaciones desde la óptica de Shannon. La teoría de la información así formulada está detrás de prácticamente toda la comunicación digital moderna. Algunos ejemplos.

- **Comunicaciones móviles.** 4G, 5G, Wi-Fi, Bluetooth: los esquemas modernos usan códigos *LDPC* y *turbo codes* que operan a tasas cercanísimas a la capacidad del canal Gaussiano blanco.
- **Almacenamiento masivo.** Discos duros, SSD, NAND-Flash, RAID, todos emplean Reed–Solomon o BCH para corregir los errores intrínsecos del medio físico.
- **Misiones espaciales.** Voyager, Cassini, Mars Rover usan *concatenated codes* (Reed–Solomon como código externo y convolucional como código interno) para sobrevivir a la atenuación de la señal a través del sistema solar.
- **Mariner 9 (1971).** La sonda fotografió Marte con 64 niveles de gris (6 bits por píxel). Cada 6-tupla se codificaba con el código Reed–Muller $RM(1, 5)$ de parámetros $[32, 6, 16]$, y se decodificaba por correlación con las 64 filas de $\pm H_{32}$. Con error por bit $p \approx 0,036$ se obtenía un error por píxel del orden de $1,4 \cdot 10^{-5}$ [4, Cap. 18].
- **Compact Disc.** El CD usa dos códigos Reed–Solomon entrelazados. Pueden corregir ráfagas de hasta $\sim 4,000$ bits consecutivos, lo que equivale a unos 2,5 mm de superficie del disco.
- **ISBN-10.** Los diez dígitos de un libro $a_1 \dots a_{10}$ deben satisfacer $\sum_{i=1}^{10} i a_i \equiv 0 \pmod{11}$. Esta única ecuación detecta cualquier error en una posición y cualquier transposición de dos dígitos.¹

¹Es precisamente la razón por la que el último dígito de un ISBN-10 puede ser “X”: representa el valor 10 mód 11.

Una digresión cariñosa. Los autores de [1] comienzan su libro con un ejemplo familiar: el hijo de uno de ellos pidió “*ee need lykle lykle mix*” en lugar de “*I need little little (trail)mix*”. La frase está plagada de errores de codificación, pero el padre la entiende. ¿Por qué? Porque el lenguaje natural está cargado de redundancia y nuestro cerebro decodifica por máxima verosimilitud sin pensarlo. La teoría de códigos formaliza ese proceso intuitivo en términos algebraicos y combinatorios.

Del azar al peor caso: Hamming (1950)

Dos años después del paper de Shannon, Richard Hamming —ingeniero en los Bell Labs— publicó un artículo [9] que cambiaría el espíritu de la disciplina. La anécdota que lo motivó es famosa.

A fines de los años cuarenta Hamming corría sus cálculos en una computadora a relés. Las corridas se ejecutaban en fines de semana, sin supervisión. Si en alguna operación se detectaba un error de paridad, el programa abortaba: el lunes Hamming se encontraba con horas de cómputo perdidas. “Si la máquina sabe que hay un error,” habría dicho, “¿por qué no es capaz de decir cuál y corregirlo?” Esa frustración devino el primer código corrector de errores no trivial *construido explícitamente*: el código de Hamming $[7, 4, 3]_2$, que veremos en detalle más adelante.

Un cambio de marco. Hamming no piensa el canal como un objeto probabilístico. Piensa los errores como una *cantidad*: el número de coordenadas donde la palabra recibida difiere de la transmitida. Y su pregunta no es “en promedio, ¿cuán bien me va?” sino “en el peor caso, ¿puedo *garantizar* la corrección?”

Formalmente, el modelo de error es:

$$\mathbf{y} = \mathbf{c} + \mathbf{e}, \quad \text{con } \text{wt}(\mathbf{e}) \leq t,$$

donde $\text{wt}(\mathbf{e})$ cuenta cuántas coordenadas de $\mathbf{e}$ son no nulas —el *peso de Hamming*, que se define con precisión más adelante—, y $t \in \mathbb{N}$ es el presupuesto de errores. Bob recibe $\mathbf{y}$ sin saber qué patrón $\mathbf{e}$ fue aplicado y quiere recuperar $\mathbf{c}$ —y por tanto $\mathbf{m}$ — *para todo* $\mathbf{e}$ con $\text{wt}(\mathbf{e}) \leq t$.

Las preguntas pasan a ser combinatorias. Bajo el marco de Hamming, no hay distribución ni asintótica. Las preguntas naturales se vuelven geométricas:

- Dado un código $\mathcal{C} \subseteq \Sigma^n$, ¿hasta cuántos errores puede corregir sin ambigüedad?
- Fijos n y t , ¿cuán grande puede ser $|\mathcal{C}|$? Equivalentemente, ¿cuál es la tasa máxima?
- ¿Existen codificaciones explícitas —no aleatorias— que alcancen esa cota?

La métrica que organiza estas preguntas es la *distancia de Hamming* entre palabras: el número de coordenadas en las que difieren. Dos codewords “lejanos” son indistinguibles bajo errores pequeños; dos cercanos pueden ser confundidos. El problema central pasa a ser *empaquetar* la mayor cantidad de codewords posibles dentro de Σ^n , manteniéndolos a distancia mutua $\geq 2t + 1$ entre sí.

Shannon vs. Hamming: dos lecturas del mismo fenómeno. Los dos marcos coexisten y se iluminan mutuamente:

- Shannon dice qué *es posible en promedio*, asintóticamente, sin exhibir códigos.
- Hamming dice qué *es posible garantizar* en el peor caso, con códigos finitos y explícitos.

Para el mismo canal $\text{BSC}(p)$, la pregunta “¿cuántos errores corrige?” tiene sentido en ambos marcos, pero con respuestas diferentes:

- *Shannon*: para $R < 1 - H_2(p)$ existe (asintóticamente) un código con $P_{\text{err}} \rightarrow 0$.
- *Hamming*: se pide garantizar corrección del peor patrón con $\text{wt}(\mathbf{e}) \leq t$; típicamente t/n debe estar por debajo de p y se requiere bloque finito.

La cota de Hamming es, en general, más pesimista: pide algo más fuerte y por eso obtiene menos. Pero entrega *garantías exactas y constructivas* (Sección 4 en adelante).

En este seminario adoptamos el marco de *Hamming*. La razón se aclara en la subsección siguiente.

Foreshadowing: códigos en ZK

Hasta acá, la motivación es la transmisión clásica: dos partes honestas que se enfrentan a un canal pasivo, sea probabilístico (Shannon) o combinatorio (Hamming). El seminario, sin embargo, no se ocupa principalmente de canales físicos: estudiamos códigos porque *aparecen, casi escondidos, en el corazón de las pruebas de conocimiento cero modernas* (zero-knowledge proofs).

El adversario en ZK. En este contexto, el “ruido” no viene de un canal físico —viene de un *prover potencialmente deshonesto*. Esto cambia el carácter del adversario:

- En Shannon, el adversario es un canal: no tiene intenciones, perturba al azar.
- En Hamming, el adversario es abstracto: un peor caso de errores con presupuesto t , sin motivo.
- En ZK, el adversario es un *actor con voluntad*: un prover que quiere convencer al verificador de algo falso.

Conviene marcar la diferencia con un adversario *criptográfico* en el sentido estricto, porque suelen confundirse:

	Adversario de Hamming	Forger criptográfico
Qué puede hacer	corromper hasta t posiciones en tránsito	construir un mensaje completo que Bob acepte como auténtico
Qué busca	— (modeliza el peor caso del canal)	que Bob crea que el mensaje vino de Alice
Defensa adecuada	códigos correctores de errores	MACs, firmas digitales
Tipo de problema	corrección de errores	autenticación e integridad

El prover deshonesto en ZK *es* un actor con motivación, pero el ataque que realiza no es “forjar un MAC”; es algo más sutil: presentar un oráculo —típicamente, las evaluaciones de un polinomio— que está *lejos* de toda palabra-código de Reed–Solomon. La defensa, por consiguiente, es de naturaleza combinatoria, no criptográfica en el sentido clásico: detectar lejanía a la palabra-código vía *proximity testing*. El protocolo FRI, que estudiaremos al final del seminario, es justamente un test de proximidad eficiente.

La arquitectura ZK que justifica todo lo demás. Concretamente:

- En las pruebas tipo **PIOP** (*polynomial interactive oracle proofs*), el probador comprime su afirmación en un *polinomio* cuyo bajo grado está garantizado por la honestidad.
- El verificador no puede leer el polinomio entero, pero puede pedir evaluaciones puntuales: pruebas de *proximidad a la palabra-código de Reed–Solomon*.

- El protocolo **FRI** es el ejemplo canónico de un *low-degree test* eficiente. Toda su construcción se apoya en propiedades combinatorias de Reed–Solomon que probaremos en esta clase.

Por eso, aunque empezamos por lo clásico, el horizonte está fijo: *Reed–Solomon*, su *distancia mínima*, y la “*Degree Mantra*” son la columna vertebral de las cuatro clases.

Pregunta guía. Dado un alfabeto de q símbolos y un bloque de longitud n , ¿cuántos mensajes distintos podemos transmitir con la garantía de corregir hasta t errores?

2. El objeto “código”

2.1. Definiciones básicas

Cualquier formalización tiene que empezar por nombrar los elementos en juego: el alfabeto, el bloque, el código.

Definición 2.1 (Alfabeto, longitud de bloque). Un *alfabeto* es un conjunto finito Σ . Llamamos $q := |\Sigma|$ a su tamaño. Una *palabra* de longitud n sobre Σ es un elemento de Σ^n , es decir una n -upla $(\sigma_1, \dots, \sigma_n)$ con $\sigma_i \in \Sigma$.

Definición 2.2 (Código). Un *código* de longitud n sobre Σ es un subconjunto $\mathcal{C} \subseteq \Sigma^n$. Sus elementos se llaman *palabras-código* (codewords).

Aquí ya conviene desglosar lo que un código *es* y lo que *hace*.

Definición 2.3 (Función de codificación, función de decodificación). Sea $|\mathcal{C}|$ el número de palabras-código. Una *función de codificación* es una biyección

$$E : \mathcal{M} \longrightarrow \mathcal{C}, \quad \mathcal{M} \cong [|\mathcal{C}|],$$

entre el conjunto de *mensajes* y el código. Una *función de decodificación* es cualquier mapa $D : \Sigma^n \rightarrow \mathcal{M}$ que recupere $\mathbf{m}$ a partir de la palabra recibida $\mathbf{y}$ (que puede tener errores).

Notación 2.4. Usaremos $\mathbf{m}$ para mensajes, $\mathbf{c}$ para palabras-código, $\mathbf{y}$ para palabras recibidas (con errores), $\mathbf{e} := \mathbf{y} - \mathbf{c}$ para el patrón de error, $\hat{\mathbf{m}}$ y $\hat{\mathbf{c}}$ para las estimaciones de Bob.

Definición 2.5 (Dimensión y tasa). La *dimensión* de un código $\mathcal{C}$ sobre alfabeto Σ de tamaño q es

$$k := \log_q |\mathcal{C}|.$$

Su *tasa* (rate) es

$$R := \frac{k}{n} = \frac{\log_q |\mathcal{C}|}{n}.$$

Nota. La definición *no* requiere que k sea entero (eso ocurrirá recién cuando el código sea lineal). La interpretación es: k es la cantidad de símbolos de Σ “de información” que se transmiten en un bloque de n símbolos. La tasa $R \in (0, 1]$ mide la fracción del bloque que “no es redundancia”. Cuanto más alta R , menos pagamos por la protección; cuanto más baja, más protegidos estamos.

2.2. Dos ejemplos de juguete que ya enseñan todo

Ejemplo 2.6 (Código de repetición $\mathcal{C}_{n,\text{rep}}$). Sea $\Sigma = \{0, 1\}$ y $n = 3$. Definimos

$$\mathcal{C}_{3,\text{rep}} := \{(b, b, b) : b \in \{0, 1\}\} = \{000, 111\}.$$

Aquí $k = \log_2 2 = 1$, $R = 1/3$. La codificación es $E(0) = 000$, $E(1) = 111$. Imaginemos que Alice manda $\mathbf{c} = 111$ y un solo bit se invierte, así que Bob recibe $\mathbf{y} = 110$. ¿Cómo decodificar? La

regla más natural es *voto por mayoría*: como $\mathbf{y}$ tiene dos unos y un cero, Bob decide $\hat{\mathbf{m}} = 1$. Esta regla es óptima frente a un canal binario simétrico siempre que la probabilidad de flip sea menor que $1/2$.

Generalización: para n impar, $\mathcal{C}_{n,\text{rep}}$ corrige hasta $\lfloor (n-1)/2 \rfloor$ errores. La tasa, sin embargo, es $1/n$: pagamos carísimo la robustez.

Ejemplo 2.7 (Código de paridad $\mathcal{C}_\oplus$). Sea $\Sigma = \{0, 1\}$, $n = 5$, $k = 4$. Codificamos $\mathbf{m} = (m_1, m_2, m_3, m_4)$ como

$$E(\mathbf{m}) = (m_1, m_2, m_3, m_4, m_1 \oplus m_2 \oplus m_3 \oplus m_4),$$

donde $\oplus$ es XOR (suma en $\mathbb{F}_2$). La quinta coordenada es la paridad de las cuatro primeras. Tasa $R = 4/5$, casi sin redundancia.

Si en transmisión se invierte un bit, la paridad falla y Bob *detecta* el error. Pero no puede *corregirlo*: no sabe en qué coordenada ocurrió. Si se invierten dos bits, la paridad vuelve a cuadrar y el error pasa desapercibido. En resumen: $\mathcal{C}_\oplus$ *detecta* cualquier número impar de errores, no *corrige* ninguno.

Nota (Tensión central). Estos dos ejemplos delimitan el espectro:

- Repetición: tasa baja, mucha protección. Detecta y corrige.
- Paridad: tasa alta, poca protección. Detecta pero no corrige.

La pregunta natural — “¿qué cualidad estructural distingue *detectar* de *corregir*, y cuántos errores puedo manejar?” — nos obliga a medir cuán parecidas son dos palabras del código.

3. La métrica de Hamming

3.1. Repaso mínimo: espacios métricos

Para hablar de “cuánto se parecen” dos palabras necesitamos una distancia. La definición general es la siguiente.

Definición 3.1 (Espacio métrico). Un *espacio métrico* es un par (X, d) donde X es un conjunto y $d : X \times X \rightarrow \mathbb{R}_{\geq 0}$ satisface, para todos $x, y, z \in X$:

M1. (Positividad) $d(x, y) \geq 0$, y $d(x, y) = 0$ si y sólo si $x = y$.

M2. (Simetría) $d(x, y) = d(y, x)$.

M3. (Desigualdad triangular) $d(x, z) \leq d(x, y) + d(y, z)$.

Ejemplo 3.2 (Métrica usual en $\mathbb{R}^n$ y un no-ejemplo). La distancia euclídea $d_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum (x_i - y_i)^2}$ es métrica. La función $d(x, y) = (x - y)^2$ en $\mathbb{R}$ *no* lo es: falla la triangular, ya que $d(0, 2) = 4 > d(0, 1) + d(1, 2) = 1 + 1 = 2$.

Nota. La pregunta “¿qué métrica le ponemos a Σ^n ?” tiene muchas respuestas. La que sigue — distancia de Hamming — es la natural cuando el modelo de error es *coordenada a coordenada*, es decir cuando no hay relación geométrica entre las posiciones (a diferencia, por ejemplo, de un canal con *ráfagas* de error donde una métrica con peso por “cluster” tendría más sentido).

3.2. Distancia de Hamming

Definición 3.3 (Distancia de Hamming). Para $\mathbf{x}, \mathbf{y} \in \Sigma^n$, la *distancia de Hamming* entre ellas es el número de coordenadas en que difieren:

$$\Delta(\mathbf{x}, \mathbf{y}) := \#\{i \in \{1, \dots, n\} : x_i \neq y_i\}.$$

La *distancia relativa* es $\delta(\mathbf{x}, \mathbf{y}) := \Delta(\mathbf{x}, \mathbf{y})/n \in [0, 1]$.

Proposición 3.4 (Hamming es métrica). (Σ^n, Δ) es un espacio métrico.

Demostración. **M1.** $\Delta(\mathbf{x}, \mathbf{y}) \geq 0$ por ser cardinal. Es 0 exactamente cuando no hay coordenadas que difieran, es decir $\mathbf{x} = \mathbf{y}$. **M2.** “ $x_i \neq y_i$ ” es simétrica en i . **M3.** Sean $\mathbf{x}, \mathbf{y}, \mathbf{z}$ y i una coordenada en que $x_i \neq z_i$. Entonces forzosamente $x_i \neq y_i$ o $y_i \neq z_i$ (si fueran iguales en ambos casos, $x_i = y_i = z_i$). Luego

$$\{i : x_i \neq z_i\} \subseteq \{i : x_i \neq y_i\} \cup \{i : y_i \neq z_i\},$$

y tomando cardinales se obtiene $\Delta(\mathbf{x}, \mathbf{z}) \leq \Delta(\mathbf{x}, \mathbf{y}) + \Delta(\mathbf{y}, \mathbf{z})$. $\square$

Definición 3.5 (Peso de Hamming). Si Σ contiene un elemento distinguido 0 (que será siempre el cero del cuerpo cuando trabajemos con códigos lineales), el *peso de Hamming* de $\mathbf{x} \in \Sigma^n$ es

$$\text{wt}(\mathbf{x}) := \#\{i : x_i \neq 0\} = \Delta(\mathbf{x}, \mathbf{0}).$$

Ejemplo 3.6. En $\mathbb{F}_2^7$, sean

$$\mathbf{x} = 1010110, \quad \mathbf{y} = 1110010, \quad \mathbf{z} = 0001001.$$

Entonces $\Delta(\mathbf{x}, \mathbf{y}) = 2$ (difieren en las posiciones 2 y 5), $\text{wt}(\mathbf{x}) = 4$, y se verifica $\Delta(\mathbf{x}, \mathbf{z}) = 5 \leq \Delta(\mathbf{x}, \mathbf{y}) + \Delta(\mathbf{y}, \mathbf{z}) = 2 + 5 = 7$. La triangular no es ajustada en general.

3.3. Distancia mínima del código

Definición 3.7 (Distancia mínima). Sea $\mathcal{C} \subseteq \Sigma^n$ con $|\mathcal{C}| \geq 2$. Su *distancia mínima* es

$$d(\mathcal{C}) := \min_{\substack{\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C} \\ \mathbf{c}_1 \neq \mathbf{c}_2}} \Delta(\mathbf{c}_1, \mathbf{c}_2).$$

La *distancia relativa* es $\delta(\mathcal{C}) := d(\mathcal{C})/n$.

Notación 3.8 (Parámetros de un código). Si $\mathcal{C} \subseteq \Sigma^n$ tiene $|\mathcal{C}| = q^k$ palabras y distancia mínima d , escribimos

$$\mathcal{C} \text{ es un } (n, k, d)_q \text{ código.}$$

Cuando además $\mathcal{C}$ es un *código lineal* (Sec. 4) usamos corchetes: $[n, k, d]_q$. Esta convención de paréntesis vs. corchetes la respetan [1, 2] y permite distinguir códigos generales de los lineales con sólo mirar la notación.

3.4. Bola de Hamming, detección y corrección

Definición 3.9 (Bola de Hamming). La *bola* centrada en $\mathbf{x} \in \Sigma^n$ de radio $r \in \mathbb{N}$ es

$$B(\mathbf{x}, r) := \{\mathbf{y} \in \Sigma^n : \Delta(\mathbf{x}, \mathbf{y}) \leq r\}.$$

Lema 3.10 (Volumen de la bola). $|B(\mathbf{x}, r)| = \sum_{i=0}^r \binom{n}{i} (q-1)^i$.

Demostración. Las palabras a distancia exactamente i de $\mathbf{x}$ se obtienen eligiendo las i coordenadas a perturbar ($\binom{n}{i}$ formas) y, en cada una, eligiendo cualquiera de los $q - 1$ valores distintos del original. $\square$

Teorema 3.11 (Detección y corrección via d). *Sea $\mathcal{C}$ un código con distancia mínima $d \geq 2$. Las siguientes afirmaciones son ciertas:*

- (1) $\mathcal{C}$ detecta hasta $d - 1$ errores.
- (2) $\mathcal{C}$ corrige hasta $\lfloor (d - 1)/2 \rfloor$ errores.
- (3) $\mathcal{C}$ recupera hasta $d - 1$ borraduras.

Más aún, las cotas de los puntos (1) y (2) son ajustadas: existen palabras recibidas con d errores que pasan inadvertidos, y existen palabras recibidas con $\lfloor (d - 1)/2 \rfloor + 1$ errores que el decodificador por mínima distancia no puede asignar de manera única (cuando d es par).

Demostración. (1) Si $\mathbf{c} \in \mathcal{C}$ se transmite y se reciben $\mathbf{y}$ con $1 \leq \Delta(\mathbf{c}, \mathbf{y}) < d$, entonces $\mathbf{y} \notin \mathcal{C}$ (porque toda otra palabra-código está al menos a distancia d de $\mathbf{c}$). Bob detecta el error al constatar que $\mathbf{y}$ no es palabra-código.

(2) Sea $t := \lfloor (d - 1)/2 \rfloor$. Afirimo que las bolas $B(\mathbf{c}, t)$ con $\mathbf{c} \in \mathcal{C}$ son disjuntas. En efecto, si $\mathbf{y} \in B(\mathbf{c}_1, t) \cap B(\mathbf{c}_2, t)$ con $\mathbf{c}_1 \neq \mathbf{c}_2$, por triangular

$$d \leq \Delta(\mathbf{c}_1, \mathbf{c}_2) \leq \Delta(\mathbf{c}_1, \mathbf{y}) + \Delta(\mathbf{y}, \mathbf{c}_2) \leq 2t \leq d - 1,$$

contradicción. Como las bolas son disjuntas, dada $\mathbf{y} \in B(\mathbf{c}, t)$ existe una única palabra-código a distancia $\leq t$, y Bob la encuentra (al menos en principio: no decimos que sea eficiente).

(3) Si Bob recibe $\mathbf{y}$ con hasta $d - 1$ posiciones borradas (símbolo desconocido), las palabras-código son distinguibles entre sí mirando sólo las $\geq n - (d - 1)$ posiciones no borradas, porque dos palabras-código difieren en al menos d coordenadas y a lo sumo $d - 1$ están borradas, así que difieren en alguna coordenada visible. $\square$

Nota. La fórmula honesta para corrección es $t = \lfloor (d - 1)/2 \rfloor$ para todo d : cuando d es par, $t = d/2 - 1$, no $d/2$. La razón: con $d = 2t + 2$, una palabra recibida puede estar simultáneamente a distancia $t + 1$ de dos palabras-código distintas (puede ubicarse en el “ecuador” entre ambas).

Ejemplo 3.12 ($\mathcal{C}_{3,\text{rep}}$: las bolas particionan $\mathbb{F}_2^3$). Tomemos el código de repetición $\mathcal{C}_{3,\text{rep}} = \{000, 111\}$ con $n = 3$, $q = 2$, $d = 3$. El Teorema 3.11 da:

- Detección: hasta $d - 1 = 2$ errores.
- Corrección: $\lfloor (d - 1)/2 \rfloor = 1$ error.
- Recuperación de borraduras: hasta $d - 1 = 2$.

El volumen de la bola unitaria (Lema 3.10) es $|B(\mathbf{x}, 1)| = \binom{3}{0} + \binom{3}{1}(q - 1) = 1 + 3 = 4$. Como hay $|\mathcal{C}| = 2$ palabras-código, la cobertura total es $2 \times 4 = 8 = 2^3 = |\Sigma^n|$. Es decir, las bolas $B(000, 1)$ y $B(111, 1)$ *particionan* $\{0, 1\}^3$: todo punto cae en exactamente una bola unitaria de un codeword. (Esta propiedad se llama *código perfecto*; volveremos sobre ella al ver el código de Hamming.) Concretamente, si Bob recibe $\mathbf{y} = 110$, entonces $\Delta(\mathbf{y}, 111) = 1$ y $\Delta(\mathbf{y}, 000) = 2$, y decodifica $\hat{\mathbf{c}} = 111$ por voto de mayoría.

Ejemplo 3.13 ($\mathcal{C}_\oplus$: detección sin corrección). Para el código de paridad $\mathcal{C}_\oplus \subseteq \mathbb{F}_2^5$ del Ejemplo 2.7, con $d = 2$, el teorema da:

- Detección: hasta $d - 1 = 1$ error.
- Corrección: $\lfloor 1/2 \rfloor = 0$ errores.

Una falla de paridad alerta a Bob de que algo cambió, pero *no* le permite localizar la coordenada errónea. Si se invierten dos bits, la paridad vuelve a cuadrar y el error pasa inadvertido. El precio de tasa alta ($R = 4/5$) es exactamente esto: detección sin corrección.

3.5. Decodificación por lista

La cota $\lfloor (d-1)/2 \rfloor$ del Teorema 3.11 caracteriza la *decodificación única*: a Bob se le exige producir *una* palabra-código consistente con $\mathbf{y}$. Esta exigencia es a veces demasiado fuerte: ¿qué pasa si los errores superan ese umbral? Para tasas R moderadas-altas, la barrera $\lfloor (d-1)/2 \rfloor$ se vuelve restrictiva, y aplicaciones modernas —en particular los tests de proximidad usados en ZK— requieren ir más allá.

La idea: relajar la unicidad. En lugar de exigir respuesta única, le permitimos a Bob devolver una *lista* (pequeña) de candidatos. Fijado un radio $\rho \in [0, 1]$, su salida es

$$\text{Lista}_\rho(\mathbf{y}) := \{ \mathbf{c} \in \mathcal{C} : \Delta(\mathbf{c}, \mathbf{y}) \leq \rho n \}.$$

La pregunta cambia: ¿hasta qué ρ podemos garantizar que la lista nunca crezca demasiado?

Definición 3.14 (Código (ρ, L) -decodificable por lista). Sea $\mathcal{C} \subseteq \Sigma^n$, $\rho \in [0, 1]$ y $L \in \mathbb{N}$. Decimos que $\mathcal{C}$ es (ρ, L) -decodificable por lista si para toda palabra recibida $\mathbf{y} \in \Sigma^n$ vale

$$|\{ \mathbf{c} \in \mathcal{C} : \Delta(\mathbf{c}, \mathbf{y}) \leq \rho n \}| \leq L.$$

El parámetro ρ es el *radio (relativo) de decodificación por lista*, y L es el *tamaño máximo de lista*.

Observación 3.15 (Decodificación única como caso $L = 1$). Si d es la distancia mínima de $\mathcal{C}$, las bolas $B(\mathbf{c}, \lfloor (d-1)/2 \rfloor)$ con $\mathbf{c} \in \mathcal{C}$ son disjuntas (Teorema 3.11 (2)). Entonces toda $\mathbf{y}$ tiene a lo sumo una palabra-código dentro de su bola y, por consiguiente, $\mathcal{C}$ es

$$\left(\frac{\lfloor (d-1)/2 \rfloor}{n}, 1 \right)\text{-decodificable por lista.}$$

Es decir: la decodificación única clásica es el caso $L = 1$.

Ejemplo 3.16 ($\mathcal{C}_{3,\text{rep}}$ bajo lista: duplicar el radio cuesta tamaño 2). Para $\mathcal{C}_{3,\text{rep}} = \{000, 111\}$ ($n = 3$, $d = 3$):

- *Único.* Radio $\rho_{\text{uniq}} = \frac{\lfloor (d-1)/2 \rfloor}{n} = \frac{1}{3}$. Para toda $\mathbf{y}$ hay a lo sumo *una* palabra-código a distancia ≤ 1 .
- *Lista en radio $\rho = \frac{2}{3}$.* Para cualquier $\mathbf{y} \in \{0, 1\}^3$ vale $\Delta(\mathbf{y}, 000) + \Delta(\mathbf{y}, 111) = 3$, así que *ambas* están a distancia ≤ 2 de $\mathbf{y}$. La lista contiene los 2 codewords. Entonces $\mathcal{C}_{3,\text{rep}}$ es $(\frac{2}{3}, 2)$ -decodificable por lista.

Hemos *duplicado el radio* (de $\frac{1}{3}$ a $\frac{2}{3}$) al precio de listas de tamaño 2 en vez de 1 —la intuición del Teorema 3.18: el régimen entre ρ_{uniq} y la capacidad $1 - R$ es accesible si Bob tolera ambigüedad acotada.

¿Por qué interesa? Si $\rho > \frac{(d-1)/2}{n}$, la lista puede tener más de un elemento, así que Bob no resuelve unívocamente. Pero *si la lista es chica* —digamos, $L = \text{poly}(n)$ — en muchas aplicaciones eso alcanza:

- En contextos interactivos o criptográficos, Bob puede usar información lateral (un *commitment*, un *challenge* aleatorio del verificador) para descartar a posteriori los candidatos espurios.
- En tests de proximidad (FRI, IOPs polinómicos) no necesitamos la palabra exacta: alcanza con saber si *existe* alguna palabra-código en una vecindad de $\mathbf{y}$ —propiedad que sí controla la decodificación por lista.

Hasta dónde se puede llegar. Dos cotas combinatorias universales organizan el panorama; ambas se desarrollan en detalle en [3].

Teorema 3.17 (Cota de Johnson). *Sea $\mathcal{C} \subseteq \Sigma^n$ con alfabeto de tamaño q y distancia relativa $\delta = d/n$. Si*

$$\rho < J_q(\delta) := \left(1 - \frac{1}{q}\right) \left(1 - \sqrt{1 - \frac{q}{q-1} \delta}\right),$$

entonces $\mathcal{C}$ es (ρ, L) -decodificable por lista con $L = O(n^2)$. En particular, sobre el límite $q \rightarrow \infty$, la cota tiende a $J(\delta) = 1 - \sqrt{1 - \delta}$.

Nota. La función $J(\delta) = 1 - \sqrt{1 - \delta}$ satisface $J(\delta) > \delta/2$ para todo $\delta \in (0, 1)$: el radio de Johnson es *estrictamente mayor* que el radio de decodificación única (alrededor de $\delta/2$ para d grande). Para $\delta \rightarrow 1$, $J(\delta) \rightarrow 1$. El régimen $\rho \in (\delta/2, J(\delta))$ es la zona donde la decodificación por lista aporta novedad combinatoria sobre la decodificación única.

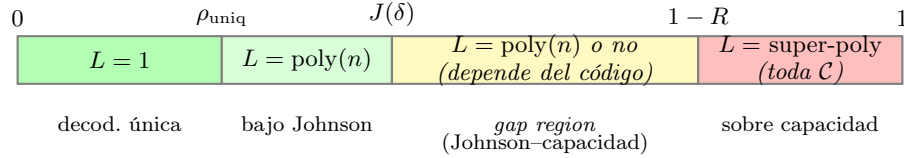
Teorema 3.18 (Capacidad de list decoding bajo errores adversariales). *Para todo $\varepsilon > 0$ existen códigos $\mathcal{C} \subseteq \Sigma^n$ con tasa R que son (ρ, L) -decodificables por lista, con*

$$\rho = 1 - R - \varepsilon, \quad L = O(1/\varepsilon).$$

Recíprocamente, si $\rho > 1 - R$, ningún código de tasa R puede ser (ρ, L) -decodificable por lista con $L = \text{poly}(n)$.

El umbral fundamental de la decodificación por lista es por lo tanto $\rho_{\text{máx}} = 1 - R$. Comparado con el radio clásico de decodificación única $\rho_{\text{unq}} \leq (1 - R)/2$, la decodificación por lista gana *un factor dos* en el radio que se sabe manejar.

¿Cuándo deja de ser polinomial la lista? La cota de Johnson y la capacidad $1 - R$ delimitan tres regímenes cualitativamente distintos para el tamaño de lista. Conviene tenerlos separados porque corresponden a obstrucciones de naturaleza muy diferente.



(1) Por encima de la capacidad ($\rho > 1 - R$): obstrucción universal.

Éste es un límite *información-teórico*: no depende del código. La intuición se ve mejor con un argumento de conteo, que desarrollamos paso a paso.

Paso 1 — la entropía q -aria. Para alfabetos q -arios necesitamos una generalización de la entropía binaria H_2 del Ejemplo 1.4.

Definición 3.19 (Entropía q -aria). Para $q \geq 2$ entero y $\rho \in [0, 1]$, la *entropía q -aria* (a veces llamada *función de entropía de Hamming*) es

$$H_q(\rho) := \rho \log_q(q - 1) - \rho \log_q \rho - (1 - \rho) \log_q(1 - \rho),$$

con la convención $0 \log 0 = 0$.

Tres propiedades inmediatas que conviene tener presentes:

- $H_q(0) = 0$ y $H_q(1) = \log_q(q - 1)$. Para $q = 2$ esto recupera $H_2(0) = H_2(1) = 0$.
- H_q es estrictamente cóncava en $[0, 1]$ y alcanza su máximo $H_q(1 - 1/q) = 1$ en $\rho^* = 1 - 1/q$ (la fracción “ruido máximo”). Es estrictamente creciente en $[0, \rho^*]$.
- Para q grande, $\log_q(q - 1) \rightarrow 1$ y los términos $-\log_q \rho, -\log_q(1 - \rho)$ tienden a 0, así que $H_q(\rho) \rightarrow \rho$. La entropía q -aria se “aplana” a la identidad cuando el alfabeto crece.

Paso 2 — el volumen de la bola en términos de H_q . Recordemos el Lema 3.10:

$$|B(\mathbf{x}, r)| = \sum_{i=0}^r \binom{n}{i} (q-1)^i.$$

Sustituyendo $r = \rho n$ con $\rho < 1 - 1/q$, esta suma queda dominada por su término más grande, que es $\binom{n}{\rho n} (q-1)^{\rho n}$. Usando la aproximación estándar de Stirling

$$\binom{n}{\rho n} \sim \rho^{-\rho n} (1-\rho)^{-(1-\rho)n} \cdot \frac{1}{\sqrt{n}}$$

y la identidad

$$q^{nH_q(\rho)} = (q-1)^{\rho n} \cdot \rho^{-\rho n} \cdot (1-\rho)^{-(1-\rho)n} \quad (\text{por la definición de } H_q),$$

obtenemos el siguiente estimado pedagógicamente clave.

Lema 3.20 (Volumen de la bola como potencia de q). *Para todo $\rho \in (0, 1 - 1/q)$,*

$$\frac{1}{\text{poly}(n)} q^{nH_q(\rho)} \leq |B(\mathbf{x}, \rho n)| \leq q^{nH_q(\rho)}.$$

En palabras: el volumen de la bola crece como $q^{nH_q(\rho)}$, módulo factores polinomiales en n . La entropía q -aria es exactamente el “ritmo exponencial” de la bola.

Paso 3 — conteo y palomar. Ahora estamos listos para la cuenta principal. Sea $\mathcal{C} \subseteq \Sigma^n$ un código de tasa R , es decir $|\mathcal{C}| = q^{nR}$, y consideremos las bolas $B(\mathbf{c}, \rho n)$ centradas en cada codeword $\mathbf{c} \in \mathcal{C}$.

- Cada bola tiene volumen $\leq q^{nH_q(\rho)}$ (Lema 3.20).
- La cantidad total de pares $(\mathbf{c}, \mathbf{y})$ con $\mathbf{c} \in \mathcal{C}$ y $\mathbf{y} \in B(\mathbf{c}, \rho n)$ es entonces

$$M := \sum_{\mathbf{c} \in \mathcal{C}} |B(\mathbf{c}, \rho n)| \leq |\mathcal{C}| \cdot q^{nH_q(\rho)} = q^{n(R+H_q(\rho))}.$$

- Por otro lado, hay solo $|\Sigma^n| = q^n$ palabras recibidas posibles $\mathbf{y} \in \Sigma^n$.

El *tamaño promedio de lista* sobre todas las $\mathbf{y} \in \Sigma^n$ es

$$\bar{L} := \frac{1}{|\Sigma^n|} \sum_{\mathbf{y} \in \Sigma^n} |\{\mathbf{c} \in \mathcal{C} : \Delta(\mathbf{c}, \mathbf{y}) \leq \rho n\}| = \frac{M}{q^n} \leq q^{n(R+H_q(\rho)-1)}.$$

Y la cota análoga por debajo (usando la otra mitad del Lema 3.20) da

$$\bar{L} \geq \frac{1}{\text{poly}(n)} q^{n(R+H_q(\rho)-1)}.$$

Paso 4 — el umbral. La condición crítica es entonces

$$\boxed{R + H_q(\rho) > 1}$$

o equivalentemente

$$\rho > H_q^{-1}(1 - R),$$

donde H_q^{-1} denota la rama creciente de la inversa en $[0, 1 - 1/q]$.

Cuando esto vale, $\bar{L}$ es exponencial en n . Como el tamaño *máximo* de lista es al menos el promedio, existe alguna palabra recibida $\mathbf{y}^* \in \Sigma^n$ con

$$|\{\mathbf{c} \in \mathcal{C} : \Delta(\mathbf{c}, \mathbf{y}^*) \leq \rho n\}| \geq \bar{L} \geq \frac{q^{n(R+H_q(\rho)-1)}}{\text{poly}(n)} = q^{\Theta(n)},$$

una lista *exponencialmente grande*. Esta $\mathbf{y}^*$ es contraejemplo a cualquier intento de decodificación por lista polinomial.

Paso 5 — por qué se escribe “ $1 - R$ ” en el enunciado del Teorema 3.18. El umbral exacto es $\rho > H_q^{-1}(1 - R)$, pero por la observación final del Paso 1 ($H_q \rightarrow \text{id}$ cuando $q \rightarrow \infty$) vale

$$H_q^{-1}(1 - R) \xrightarrow{q \rightarrow \infty} 1 - R,$$

y para q moderadamente grande la aproximación $H_q^{-1}(1 - R) \approx 1 - R$ es muy buena. En el régimen criptográfico, donde q es del orden del primo del cuerpo $\mathbb{F}_q$ (típicamente $\geq 2^{60}$), la diferencia es despreciable. Por eso $\rho_{\text{máx}} = 1 - R$ es el umbral que aparece en el Teorema 3.18 y en la literatura.

Conclusión. Por encima de $\rho = H_q^{-1}(1 - R) \approx 1 - R$, el espacio de Hamming *obliga* a que alguna palabra recibida tenga lista exponencial. Esta obstrucción es *estructural al espacio de Hamming*, no al código particular. **Ningún código de tasa R puede tener lista polinomial más allá de $1 - R$.**

(2) Entre Johnson y capacidad: obstrucción específica al código.

En el rango $\rho \in (J(\delta), 1 - R)$ —la *gap region*— la respuesta deja de ser universal y pasa a *dependen del código*:

- Para códigos *bien estructurados* —códigos aleatorios, códigos de Algebraic-Geometry (AG codes), RS sobre puntos de evaluación “genéricos” o aleatorios— la lista *permanece polinomial* todo el camino hasta capacidad. Esa es la parte de existencia del Teorema 3.18: hay códigos de tasa R que son $(1 - R - \varepsilon, O(1/\varepsilon))$ -decodificables por lista. Pero la prueba es *existencial* (códigos aleatorios via método probabilístico), no constructiva en general.
- Para códigos *patológicos*, la lista puede crecer super-polinomialmente apenas se cruza Johnson. Ben-Sasson, Kopparty y Saraf, y más recientemente [10], exhibieron familias específicas de RS donde justamente en $\rho \gtrsim J(\delta)$ el tamaño de lista crece como $n^{\Theta(1)}$ y, para radios apenas más allá, se vuelve super-poly. Esto se traduce en *obstrucciones cuantitativas* al fenómeno de proximity gaps por encima de Johnson —que ya mencionamos en el horizonte de [10].

La conjetura “los RS son list-decodable hasta capacidad” es por lo tanto un teorema *específico sobre los RS*, no una propiedad universal. Hasta la fecha sigue abierta para RS generales, con resultados recientes que la sostienen sobre puntos de evaluación aleatorios pero la refutan en su forma más fuerte sobre puntos arbitrarios.

Por qué Johnson es un techo “blando”.

Una intuición útil: $J(\delta)$ no es donde las listas se vuelven exponenciales —es donde *deja de existir una prueba uniforme* de que sean polinomiales. La cota de Johnson es combinatoria-pura: se aplica a cualquier código con distancia mínima d , sin usar más estructura. Para ir más allá hay que mirar la estructura algebraica específica del código. Por eso los resultados positivos para RS entre Johnson y capacidad requieren los algoritmos de Guruswami–Sudan (técnica algebraica), y por eso son tan difíciles de extender hasta $1 - R$ —se está pidiendo, en cierta forma, una propiedad que vive en la frontera entre lo combinatorio y lo algebraico.

Algorítmica vs. combinatoria. La discusión anterior es *combinatoria*: “existe” una decodificación por lista con esos parámetros. La pregunta algorítmica es: dado $\mathbf{y}$, ¿hay un algoritmo eficiente que *construya* la lista? La respuesta no es automática y depende del código. Para códigos generales no se conoce; para códigos estructurados sí:

- **Sudan (1997).** Algoritmo polinomial que decodifica $\text{RS}_q[\alpha, k]$ por lista hasta $\rho < 1 - \sqrt{2R}$, con $R = k/n$ [3].
- **Guruswami–Sudan (1999).** Mejora la cota a la barrera de Johnson: $\rho < 1 - \sqrt{R}$. Es el algoritmo de referencia para RS.

Estos algoritmos producen *constructivamente* la lista y son el corazón de los tests de bajo grado (*low-degree testing*) modernos.

Por qué importa para este seminario. El protocolo FRI —y en general toda prueba PIOP que invoque proximidad a RS— vive justamente en este régimen: el verificador no exige que el oráculo sea *exactamente* un codeword de RS, sino que esté ρ -cerca de uno. La elección óptima del radio ρ requiere entender la cota de Johnson y la capacidad de list decoding. Volveremos sobre esto en las clases posteriores.

Mirando al horizonte: proximity gaps y low-degree testing. La decodificación por lista responde la pregunta “¿hay codewords cerca de y ?”. Para las aplicaciones criptográficas modernas se necesita una propiedad más fina, que organiza la *soundness* de los tests de proximidad: dadas dos palabras $f, g \in \Sigma^n$, ¿cuántas combinaciones lineales $f + z g$ (con $z \in \mathbb{F}_q$) deben estar δ -cerca del código antes de garantizar que *ambas* f y g lo están? Ese fenómeno, llamado *proximity gap*, dice que no hay punto intermedio: o bien casi todas las combinaciones están cerca, o bien casi ninguna. Es la propiedad que sostiene la *soundness* de FRI, STIR y WHIR. Trabajos recientes [10] dan tanto cotas superiores muy ajustadas hasta el radio de Johnson como *obstrucciones cuantitativas* más allá. La cota de Johnson, que en esta subsección apareció como límite de la lista polinomial, reaparece allí como límite estructural del fenómeno de gaps. Las clases posteriores volverán sobre esto cuando analicemos FRI.

La versión multivariada: low-degree testing. El análogo multivariado del test de proximidad es el *low-degree test*: dado $f : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ accedido por oráculo, decidir si está cerca de algún polinomio multivariado de grado $\leq d$ comparando la restricción de f a líneas aleatorias contra polinomios univariados. El análisis ajustado en el régimen *high-error* sobre campos de tamaño lineal en el grado ($q = O(d)$) es reciente: véase Harsha-Kumar-Saptharishi-Sudan [11]. Este insumo es el que sostiene los PCPs eficientes que aparecerán más adelante en el seminario.

3.6. Modelos de error: Hamming vs. Shannon

- **Modelo Hamming.** El adversario introduce hasta t errores en posiciones cualesquiera; debemos garantizar decodificación *para el peor caso*.
- **Modelo Shannon.** Cada coordenada se altera de forma independiente con cierta probabilidad; analizamos la *probabilidad* de error.

En este seminario, mientras no se diga lo contrario, el modelo es *Hamming* (peor caso). Es el adecuado para nuestra agenda criptográfica: el “adversario” es un probador potencialmente malicioso, no un canal estadístico.

Pregunta guía. Fijados n, q, d , ¿cuán grande puede ser $|\mathcal{C}|$? Equivalentemente, ¿cuál es la *tasa máxima* de un código que corrige t errores?

4. De códigos cualesquiera a códigos lineales

4.1. Por qué pedir estructura

Un código arbitrario $\mathcal{C} \subseteq \Sigma^n$ es difícil de describir: en el peor caso, hay que listar sus $|\mathcal{C}|$ palabras una por una. Si $|\mathcal{C}|$ es exponencial en n (que es el régimen de interés), esto es inviable. Necesitamos estructura algebraica para dos cosas:

1. Describir el código en espacio razonable (matrices generadora / paridad).

2. Diseñar algoritmos de decodificación con costo polinomial.

La estructura mínima útil es la *lineal*, que requiere que Σ sea un cuerpo. Para que tenga sentido y el código sea finito, tomaremos $\Sigma = \mathbb{F}_q$ con q potencia de un primo.

4.2. Repaso mínimo: cuerpos finitos

Para no detenernos, recordemos sólo lo indispensable.

Teorema 4.1 (Existencia y unicidad de cuerpos finitos). *Para cada potencia de primo $q = p^s$ existe, salvo isomorfismo, un único cuerpo con q elementos. Lo denotamos $\mathbb{F}_q$.*

Ejemplo 4.2 ($\mathbb{F}_p$ con p primo). Es $\mathbb{Z}/p\mathbb{Z} = \{0, 1, 2, \dots, p-1\}$ con suma y producto módulo p . Por ejemplo $\mathbb{F}_5 = \{0, 1, 2, 3, 4\}$ con $3 \cdot 4 = 12 \equiv 2 \pmod{5}$. La inversa multiplicativa de 3 es 2, ya que $3 \cdot 2 = 6 \equiv 1$.

Ejemplo 4.3 ($\mathbb{F}_4$ con $q = 2^2$). $\mathbb{F}_4 \neq \mathbb{Z}/4\mathbb{Z}$ (este último no es cuerpo: $2 \cdot 2 = 0$ pero $2 \neq 0$). Una construcción concreta es $\mathbb{F}_4 = \mathbb{F}_2[\alpha]/(\alpha^2 + \alpha + 1)$, con elementos $\{0, 1, \alpha, \alpha + 1\}$ y la relación $\alpha^2 = \alpha + 1$.

Nota. Para esta clase no necesitamos construir $\mathbb{F}_q$ con q no primo: nos alcanza con saber que existe y operar abstractamente. La construcción explícita aparecerá cuando hablemos de Reed–Solomon sobre $\mathbb{F}_{2^m}$ (BCH binarios) en clases posteriores.

4.3. Códigos lineales

Definición 4.4 (Código lineal). Un código $\mathcal{C} \subseteq \mathbb{F}_q^n$ es *lineal* si es un subespacio vectorial de $\mathbb{F}_q^n$. Si $\mathcal{C}$ tiene dimensión k como $\mathbb{F}_q$ -espacio vectorial y distancia mínima d , decimos que es un código $[n, k, d]_q$.

Notar que entonces $|\mathcal{C}| = q^k$, así que la dimensión vectorial coincide con la dimensión del Definición 2.5.

Ejemplo 4.5 (Repaso de los anteriores). $\mathcal{C}_{3,\text{rep}} = \{000, 111\}$ es lineal (subespacio de dimensión 1 en $\mathbb{F}_2^3$). $\mathcal{C}_\oplus$ es lineal (subespacio de dimensión 4 en $\mathbb{F}_2^5$), porque la condición “suma de coordenadas = 0” es lineal.

4.4. La identidad clave: $d = \text{mín peso}$

Lema 4.6 (Distancia = peso mínimo). *Si $\mathcal{C}$ es lineal, entonces*

$$d(\mathcal{C}) = \min_{\mathbf{c} \in \mathcal{C} \setminus \{\mathbf{0}\}} \text{wt}(\mathbf{c}).$$

Demostración. Sean $\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}$ con $\mathbf{c}_1 \neq \mathbf{c}_2$. Por linealidad $\mathbf{c}_1 - \mathbf{c}_2 \in \mathcal{C}$ y es no nulo. Además

$$\Delta(\mathbf{c}_1, \mathbf{c}_2) = \#\{i : c_{1,i} - c_{2,i} \neq 0\} = \text{wt}(\mathbf{c}_1 - \mathbf{c}_2).$$

Tomando el mínimo a izquierda y derecha: el mínimo de Δ es a lo sumo el mínimo de wt (tomando $\mathbf{c}_2 = \mathbf{0}$, que pertenece al código), y el mínimo de wt es a lo sumo el mínimo de Δ (porque cada palabra no nula es alguna $\mathbf{c}_1 - \mathbf{c}_2$). Igualdad. $\square$

Nota (Por qué esto es importante). Calcular d para un código *general* de $|\mathcal{C}| = M$ palabras requiere examinar $\binom{M}{2}$ pares: cuadrático. Para uno *lineal* basta examinar $M - 1$ palabras (todas las no nulas): lineal en M . Esta reducción de complejidad combinatoria es la primera ganancia tangible de pedir estructura.

4.5. Matriz generadora y matriz de paridad

Como subespacio de dimensión k , $\mathcal{C}$ admite una base $\mathbf{g}_1, \dots, \mathbf{g}_k$. Apilando estos vectores como filas obtenemos la *matriz generadora*.

Definición 4.7 (Matriz generadora G). Una matriz $G \in \mathbb{F}_q^{k \times n}$ es *generadora* de $\mathcal{C}$ si sus filas son una base de $\mathcal{C}$, equivalentemente

$$\mathcal{C} = \{\mathbf{m}G : \mathbf{m} \in \mathbb{F}_q^k\}.$$

La codificación natural es entonces $E(\mathbf{m}) := \mathbf{m}G$.

Por dualidad (núcleo vs. imagen), $\mathcal{C}$ se puede describir también como el conjunto de vectores que satisfacen ciertas ecuaciones lineales.

Definición 4.8 (Matriz de paridad H). Una matriz $H \in \mathbb{F}_q^{(n-k) \times n}$ es *de paridad* de $\mathcal{C}$ si sus filas son una base del subespacio dual:

$$\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_q^n : H\mathbf{c}^\top = \mathbf{0}\}.$$

Por construcción $GH^\top = 0_{k \times (n-k)}$. Una y otra describen el mismo código pero desde puntos de vista complementarios: G “escribe” el código (parametrización), H “lo testea” (ecuaciones).

Proposición 4.9 (Distancia via columnas de H). Sea $\mathcal{C}$ un $[n, k, d]_q$ código con matriz de paridad H . Entonces d es el menor número de columnas de H que son linealmente dependientes.

Demostración. Por el Lema 4.6, d es el mínimo peso de un $\mathbf{c} \neq \mathbf{0}$ con $H\mathbf{c}^\top = \mathbf{0}$. Pero $H\mathbf{c}^\top$ es la combinación lineal $\sum_{i:c_i \neq 0} c_i \mathbf{h}_i$ donde $\mathbf{h}_i$ es la i -ésima columna de H . Que esta combinación sea $\mathbf{0}$ con $\mathbf{c} \neq \mathbf{0}$ equivale a tener una dependencia lineal entre las columnas $\mathbf{h}_i$ tales que $c_i \neq 0$. El mínimo peso de tal $\mathbf{c}$ es el menor tamaño de una subfamilia linealmente dependiente. $\square$

4.6. Decodificación por síndrome

Si Bob recibe $\mathbf{y} = \mathbf{c} + \mathbf{e}$, calcula

$$\mathbf{s} := H\mathbf{y}^\top = H\mathbf{c}^\top + H\mathbf{e}^\top = H\mathbf{e}^\top \in \mathbb{F}_q^{n-k}.$$

El vector $\mathbf{s}$ se llama *síndrome* y depende sólo del patrón de error $\mathbf{e}$, no de la palabra-código transmitida. La estrategia: hallar un $\mathbf{e}$ de peso mínimo compatible con el síndrome.

4.7. El ejemplo dorado: el código de Hamming $[7, 4, 3]_2$

Ejemplo 4.10 (Hamming(7,4)). Definamos $H \in \mathbb{F}_2^{3 \times 7}$ cuyas *columnas* sean las representaciones binarias (en 3 bits) de los enteros 1, 2, 3, 4, 5, 6, 7, ordenadas:

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

La i -ésima columna de H es la representación binaria del entero i . El código asociado es

$$\mathcal{C}_H := \{\mathbf{c} \in \mathbb{F}_2^7 : H\mathbf{c}^\top = \mathbf{0}\}.$$

Tiene dimensión $k = 7 - 3 = 4$, así que $|\mathcal{C}_H| = 16$. La tasa es $R = 4/7$.

Distancia. Por la Proposición 4.9, d es el menor número de columnas linealmente dependientes. Dos columnas distintas son linealmente independientes (porque son binarias distintas y no nulas, y en $\mathbb{F}_2$ las dependencias se reducen a iguales). Tres columnas pueden ser dependientes: por ejemplo las columnas 1, 2 y 3 son 001, 010, 011 y suman 000 = $\mathbf{0}$. Por lo tanto $d = 3$.

Decodificación. Supongamos que Bob recibe $\mathbf{y} = (1, 0, 1, 1, 0, 0, 1)$ y sabe que se cometió a lo sumo un error. Calcula

$$H\mathbf{y}^\top = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}.$$

La columna 011 = 3 en binario, es decir el síndrome “apunta” a la posición 3. Bob corrige la tercera coordenada: $\hat{\mathbf{c}} = (1, 0, 0, 1, 0, 0, 1)$. Verificación: $H\hat{\mathbf{c}}^\top = \mathbf{0}$ (ejercicio).

La belleza. El truco proviene de cómo se eligió H : la columna i es exactamente i en binario. Si el error ocurre en la posición i , entonces $\mathbf{e} = \mathbf{e}_i$ (vector canónico) y $H\mathbf{e}^\top = \mathbf{h}_i = i$ en binario. El síndrome *es* la dirección del error.

Optimalidad. El código de Hamming $[7, 4, 3]_2$ es *perfecto*: las bolas de radio 1 alrededor de las 16 palabras-código son disjuntas y cubren *exactamente* todo $\mathbb{F}_2^7$. En efecto, $|B(\mathbf{c}, 1)| = 1 + 7 = 8$ y $16 \cdot 8 = 128 = 2^7$. No queda ni una sola palabra recibida sin asignar.

Ejemplo 4.11 (Generalización: Hamming binarios $[2^r - 1, 2^r - 1 - r, 3]_2$). Tomar H_r como la matriz $r \times (2^r - 1)$ cuyas columnas son los $2^r - 1$ vectores no nulos de $\mathbb{F}_2^r$. El argumento del ejemplo anterior funciona idéntico: dos columnas distintas son LI, hay tres dependientes (suma de cualesquiera dos da una tercera columna por completitud), así que $d = 3$. El código es perfecto. Para $r = 3$ recuperamos el caso $[7, 4, 3]_2$; para $r = 4$ obtenemos $[15, 11, 3]_2$, etc.

Observación 4.12. Hamming corrige *un* error y nada más. Si queremos corregir más errores con código binario, el camino son los códigos BCH (subcódigos binarios de Reed–Solomon sobre $\mathbb{F}_{2^m}$). Eso será materia de la segunda mitad del seminario.

5. Códigos de Reed–Solomon

5.1. El instrumento: polinomios sobre un cuerpo

Vamos a construir códigos a partir de *evaluaciones de polinomios*. Recordemos brevemente lo necesario.

Definición 5.1 (Anillo de polinomios $\mathbb{F}_q[X]$). $\mathbb{F}_q[X]$ es el conjunto de expresiones $f(X) = \sum_{i=0}^N a_i X^i$ con $a_i \in \mathbb{F}_q$, equipado con suma y producto usuales. El *grado* de f , denotado $\deg f$, es el mayor índice i con $a_i \neq 0$ (con la convención $\deg 0 = -\infty$).

El siguiente resultado, conocido en cuerpos de característica 0 como una consecuencia inmediata del Teorema Fundamental del Álgebra y en general como “Degree Mantra” [1], es la pieza algebraica central de toda esta clase.

Teorema 5.2 (Cota del número de raíces). Sea $\mathbb{F}$ un cuerpo y $f \in \mathbb{F}[X]$ un polinomio no nulo de grado $\deg f = D$. Entonces f tiene a lo sumo D raíces en $\mathbb{F}$.

Demostración. Por inducción en D . Si $D = 0$ entonces f es una constante no nula, sin raíces. Si $D \geq 1$ y $\alpha \in \mathbb{F}$ es raíz de f , división euclidiana en $\mathbb{F}[X]$ da $f(X) = (X - \alpha) \cdot g(X)$ con $\deg g = D - 1$. Por hipótesis inductiva g tiene a lo sumo $D - 1$ raíces, así que f tiene a lo sumo D raíces. $\square$

Nota (Estatus del enunciado). Sobre $\mathbb{C}$, el TFA garantiza la *existencia* de raíces y por iteración la factorización completa. Sobre cuerpos arbitrarios la existencia puede fallar (e.g. $X^2 + 1$ no tiene raíces en $\mathbb{R}$ ni en $\mathbb{F}_3$), pero la *cota* “ $\leq D$ raíces” siempre vale, y es lo único que necesitamos. En toda esta clase, cuando hablemos de TFA nos referiremos a esta versión de cota: *un polinomio no nulo de grado D tiene a lo sumo D raíces en su cuerpo de coeficientes*.

5.2. Construcción del código

Definición 5.3 (Código de Reed–Solomon). Sean:

- $\mathbb{F}_q$ un cuerpo finito,
- n, k enteros con $1 \leq k \leq n \leq q$,
- $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)$ una n -upla de elementos *distintos* de $\mathbb{F}_q$ (los *puntos de evaluación*).

El *código de Reed–Solomon* con esos parámetros es

$$\text{RS}_q[\alpha, k] := \{ (f(\alpha_1), f(\alpha_2), \dots, f(\alpha_n)) : f \in \mathbb{F}_q[X], \deg f < k \} \subseteq \mathbb{F}_q^n.$$

La codificación natural envía $\mathbf{m} = (m_0, m_1, \dots, m_{k-1})$ al polinomio $f_{\mathbf{m}}(X) := \sum_{i=0}^{k-1} m_i X^i$ y luego a $(f_{\mathbf{m}}(\alpha_1), \dots, f_{\mathbf{m}}(\alpha_n))$.

Nota (Sobre la condición $n \leq q$). Necesitamos n elementos distintos en $\mathbb{F}_q$, así que forzosamente $n \leq q$. La elección más usada es $n = q - 1$ con α enumerando $\mathbb{F}_q^* = \mathbb{F}_q \setminus \{0\}$; o bien $n = q$ enumerando todo $\mathbb{F}_q$.

5.3. El teorema central

Teorema 5.4 (Parámetros de Reed–Solomon). $\text{RS}_q[\alpha, k]$ es un código lineal $[n, k, n - k + 1]_q$.

Demostración. Linealidad. El conjunto $\mathbb{F}_q[X]_{<k} := \{f \in \mathbb{F}_q[X] : \deg f < k\}$ es un $\mathbb{F}_q$ -espacio vectorial de dimensión k (con base $\{1, X, X^2, \dots, X^{k-1}\}$). El mapa de evaluación

$$\text{ev}_{\alpha} : \mathbb{F}_q[X]_{<k} \longrightarrow \mathbb{F}_q^n, \quad f \mapsto (f(\alpha_1), \dots, f(\alpha_n))$$

es lineal (la suma y el producto por escalar de polinomios se evalúan coordenada a coordenada). Su imagen $\text{RS}_q[\alpha, k]$ es un subespacio.

Injectividad y dimensión. Supongamos $\text{ev}_{\alpha}(f) = \mathbf{0}$. Entonces f se anula en los n puntos distintos $\alpha_1, \dots, \alpha_n$. Como $\deg f < k \leq n$, por el Teorema 5.2 el polinomio f es el polinomio nulo. Luego ev_{α} es inyectivo, y $\dim \text{RS}_q[\alpha, k] = k$.

Distancia (cota inferior). Sean $\mathbf{c}_1 = \text{ev}_{\alpha}(f_1)$ y $\mathbf{c}_2 = \text{ev}_{\alpha}(f_2)$ dos palabras-código distintas, así que $f_1 \neq f_2$. El polinomio $f_1 - f_2$ es no nulo y $\deg(f_1 - f_2) < k$. Por el Teorema 5.2 tiene a lo sumo $k - 1$ raíces. Por lo tanto, f_1 y f_2 *coinciden* en a lo sumo $k - 1$ de los puntos α_i , y *difieren* en al menos $n - (k - 1) = n - k + 1$. Así

$$\Delta(\mathbf{c}_1, \mathbf{c}_2) \geq n - k + 1.$$

Distancia (cota superior). La cota es ajustada. En efecto, el polinomio

$$g(X) := \prod_{i=1}^{k-1} (X - \alpha_i)$$

tiene grado exactamente $k - 1 < k$, se anula en los puntos $\alpha_1, \dots, \alpha_{k-1}$ y, por la elección distinta de los α_i , es no nulo en los restantes $\alpha_k, \dots, \alpha_n$. Su codeword $\mathbf{c} = \text{ev}_{\alpha}(g)$ tiene exactamente $n - (k - 1) = n - k + 1$ coordenadas no nulas. Tomando $\mathbf{c}_1 = \mathbf{0}$ (codeword del polinomio nulo) y $\mathbf{c}_2 = \mathbf{c}$ obtenemos $\Delta(\mathbf{c}_1, \mathbf{c}_2) = n - k + 1$. Concluimos $d = n - k + 1$. $\square$

5.4. Una matriz generadora explícita

Proposición 5.5 (RS y la matriz de Vandermonde). *Una matriz generadora de $\text{RS}_q[\alpha, k]$ es*

$$G = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ \alpha_1 & \alpha_2 & \cdots & \alpha_n \\ \alpha_1^2 & \alpha_2^2 & \cdots & \alpha_n^2 \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_1^{k-1} & \alpha_2^{k-1} & \cdots & \alpha_n^{k-1} \end{pmatrix} \in \mathbb{F}_q^{k \times n}.$$

Demostración. Es la matriz de la aplicación ev_α respecto a la base $\{1, X, \dots, X^{k-1}\}$ del dominio. La fila i -ésima es $\text{ev}_\alpha(X^{i-1}) = (\alpha_1^{i-1}, \dots, \alpha_n^{i-1})$. La codificación de $\mathbf{m}$ es $\mathbf{m}G$. $\square$

Nota (Conexión con álgebra lineal clásica). Cualquier submatriz $k \times k$ de G formada eligiendo k columnas distintas es una matriz de Vandermonde con k parámetros distintos, y por lo tanto es invertible (su determinante es $\prod_{i < j} (\alpha_{j_i} - \alpha_{j_m}) \neq 0$). Esto da una lectura combinatoria de la distancia: se puede recuperar f a partir de *cualquier* k evaluaciones, equivalentemente, se pueden borrar $n - k$ posiciones cualesquiera y aún reconstruir el mensaje. Esa es justamente la idea que se usará en la decodificación de borraduras.

5.5. Un ejemplo numérico completo

Ejemplo 5.6 (RS sobre $\mathbb{F}_5$ con $n = 4$, $k = 2$). Tomemos $\mathbb{F}_5 = \{0, 1, 2, 3, 4\}$ y $\alpha = (1, 2, 3, 4)$. Como $k = 2$, los polinomios admisibles son $f(X) = m_0 + m_1X$ con $m_0, m_1 \in \mathbb{F}_5$. Hay 25 mensajes.

La matriz generadora es

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{pmatrix}.$$

Veamos algunas codificaciones:

- $\mathbf{m} = (3, 0)$ corresponde a $f(X) = 3$. Codificación: $(3, 3, 3, 3)$.
- $\mathbf{m} = (0, 1)$ corresponde a $f(X) = X$. Codificación: $(1, 2, 3, 4)$.
- $\mathbf{m} = (4, 2)$ corresponde a $f(X) = 4 + 2X$. Evaluación: $f(1) = 6 \equiv 1$, $f(2) = 8 \equiv 3$, $f(3) = 10 \equiv 0$, $f(4) = 12 \equiv 2$. Codificación: $(1, 3, 0, 2)$.

Distancia. El teorema dice $d = 4 - 2 + 1 = 3$. Verifiquemos: si $f = m_0 + m_1X$ es no nulo, su grado es 0 o 1, así que tiene a lo sumo 1 raíz en $\mathbb{F}_5$. Por lo tanto $\mathbf{c} \neq \mathbf{0}$ tiene a lo sumo 1 coordenada nula, y $\text{wt}(\mathbf{c}) \geq 4 - 1 = 3$.

Decodificación de borraduras. Supongamos que Bob recibe $(?, 3, ?, 2)$ con dos borraduras (en posiciones 1 y 3) y sabe que vino de un RS. Resuelve el sistema $f(2) = 3$, $f(4) = 2$:

$$\begin{cases} m_0 + 2m_1 = 3 \\ m_0 + 4m_1 = 2 \end{cases} \Rightarrow 2m_1 = -1 \equiv 4 \Rightarrow m_1 = 2, m_0 = 4.$$

Entonces $f(X) = 4 + 2X$, y reconstruye las coordenadas borradas: $f(1) = 1$, $f(3) = 0$. La palabra original era $(1, 3, 0, 2)$, exactamente la del ejemplo anterior.

Ejemplo 5.7 (El mismo RS bajo decodificación por lista: Johnson duplica el radio). Mantengamos $\text{RS}_5[(1, 2, 3, 4), 2]$ del Ejemplo 5.6, con parámetros $[4, 2, 3]_5$ y $\delta = 3/4$. Ahora preguntamos: ¿cuántos errores tolera bajo *decodificación por lista* (Sección 3.5)?

Cálculo del radio de Johnson. Con $q = 5$ y $\delta = 3/4$, la fórmula $J_q(\delta) = (1 - \frac{1}{q})(1 - \sqrt{1 - \frac{q}{q-1}\delta})$ del Teorema 3.17 da

$$J_5(\frac{3}{4}) = \frac{4}{5} \left(1 - \sqrt{1 - \frac{5}{4} \cdot \frac{3}{4}}\right) = \frac{4}{5} \left(1 - \sqrt{\frac{1}{16}}\right) = \frac{4}{5} \cdot \frac{3}{4} = \frac{3}{5}.$$

Entonces $\rho < \frac{3}{5}$ garantiza lista de tamaño polinomial. Como $\rho n < \frac{3}{5} \cdot 4 = \frac{12}{5} = 2,4$, el código tolera hasta 2 errores por decodificación por lista —justo el doble de los $\lfloor (d-1)/2 \rfloor = 1$ que tolera la decodificación única.

Instancia concreta. Consideremos los polinomios $f_1(X) = 1 + 2X$ y $f_2(X) = 2 + X$ en $\mathbb{F}_5[X]_{<2}$, con codewords

$$\mathbf{c}_1 = (3, 0, 2, 4), \quad \mathbf{c}_2 = (3, 4, 0, 1).$$

Se verifica $\Delta(\mathbf{c}_1, \mathbf{c}_2) = 3 = d$. Supongamos que Bob recibe $\mathbf{y} = (3, 0, 0, 4)$.

- $\Delta(\mathbf{y}, \mathbf{c}_1) = 1$ (solo difiere la posición 3).
- $\Delta(\mathbf{y}, \mathbf{c}_2) = 2$ (difieren las posiciones 2 y 4).

Ambos están dentro del radio de Johnson. La lista en radio 2 contiene al menos $\{\mathbf{c}_1, \mathbf{c}_2\}$ —dos candidatos genuinos—. Bajo decodificación única (radio 1) solo aparecería $\mathbf{c}_1$, así que ni siquiera detectaríamos que $\mathbf{c}_2$ es plausible. Bob necesita información adicional —típicamente un *commitment* previo o un *challenge* aleatorio en el contexto interactivo— para descartar el espurio.

Ejemplo 5.8 (RS binario *no* es directamente útil). ¿Por qué no construir Reed–Solomon sobre $\mathbb{F}_2$? Porque la condición $n \leq q = 2$ obliga a $n \leq 2$, así que el código es trivial. Para tener RS con n moderado-grande hace falta cuerpo grande, lo cual implica símbolos no binarios. Reconciliarlo con un alfabeto binario lleva, por ejemplo, a los códigos BCH (subcódigo binario de un RS sobre $\mathbb{F}_{2^m}$). Esa será una de las construcciones que veremos más adelante.

5.6. Reed–Solomon como “el mejor código posible”

[1] titulan su capítulo de RS *The greatest code of them all*. Las razones son:

1. Su distancia $d = n - k + 1$ es la *máxima posible* para un código de longitud n y dimensión k —resultado que formalizaremos en la Clase 2 como la *cota de Singleton*, y que define la familia de los códigos *MDS* (Maximum Distance Separable).
2. Toda k coordenadas determinan el codeword (por Vandermonde).
3. Son lineales, así que admiten codificación y decodificación de borraduras eficiente.
4. Su distancia es *exactamente predecible* por el grado del polinomio.

El “precio” es el alfabeto: $q \geq n$, lo que eventualmente impacta en complejidad de bit-level.

Pregunta guía (Puente a la Clase 2). Hemos visto que RS corrige hasta $\lfloor (n-k)/2 \rfloor$ errores en principio (Teorema 3.11). Pero ¿cómo? Es decir: dado $\mathbf{y} = \mathbf{c} + \mathbf{e}$ con $\text{wt}(\mathbf{e}) \leq \lfloor (n-k)/2 \rfloor$, ¿hay un algoritmo de tiempo polinomial que recupere $\mathbf{c}$? La respuesta es *sí*, vía el *algoritmo de Berlekamp–Welch*, y a eso dedicaremos buena parte del segundo encuentro.

A. Apéndice: TFA, palabras de código y los parámetros (n, k, d, R)

Esta sección reúne, en una sola página, la interacción entre el Teorema Fundamental del Álgebra (en su forma que importa: número de raíces vs. grado) y los parámetros estructurales de un código basado en evaluación de polinomios.

A.1. El esqueleto algebraico

Sea $\mathbb{F}$ un cuerpo. Los protagonistas son tres:

- Un **polinomio** $f \in \mathbb{F}[X]$ de grado $\deg f$.
- Un **conjunto de evaluación** $\{\alpha_1, \dots, \alpha_n\} \subseteq \mathbb{F}$ con elementos distintos.
- La **palabra-código** resultante: $\mathbf{c} = (f(\alpha_1), \dots, f(\alpha_n))$.

El *Teorema Fundamental del Álgebra* (en la forma del Teorema 5.2) afirma que un polinomio no nulo de grado D tiene a lo sumo D raíces. Esta única afirmación tiene cuatro consecuencias inmediatas que articulan toda la construcción de Reed–Solomon.

A.2. Cuatro consecuencias en cadena

- (I) **Inyectividad de la evaluación.** Si $\deg f < n$ y f se anula en n puntos distintos, entonces $f \equiv 0$. Por lo tanto, fijado $k \leq n$, el mapa

$$\text{ev} : \{f \in \mathbb{F}[X] : \deg f < k\} \longrightarrow \mathbb{F}^n$$

es *inyectivo*. Su imagen es un subespacio de dimensión k .

- (II) **Dimensión = grado disponible + 1.** La cantidad de polinomios disponibles es $|\mathbb{F}|^k$, donde k es la cantidad de coeficientes libres de f , igual a $\deg f + 1$ máximo. Es decir:

$$\boxed{k = \text{grado máximo permitido} + 1}$$

Así, “ $\deg f < k$ ” es lo mismo que “ f tiene a lo sumo k coeficientes”.

- (III) **Distancia mínima** $= n - k + 1$. Dos polinomios distintos f_1, f_2 con $\deg(f_1 - f_2) < k$ tienen $f_1 - f_2$ no nulo de grado $\leq k - 1$, y por TFA esto da a lo sumo $k - 1$ raíces. Luego sus evaluaciones coinciden en a lo sumo $k - 1$ posiciones y difieren en al menos $n - k + 1$. La cota es ajustada: el polinomio $g(X) = \prod_{i=1}^{k-1} (X - \alpha_i)$ tiene grado $k - 1$ y se anula exactamente en $\alpha_1, \dots, \alpha_{k-1}$, dando un codeword de peso $n - k + 1$. En conclusión:

$$\boxed{d = n - k + 1}$$

- (IV) **Tasa = información por símbolo.**

$$R = \frac{k}{n} = \frac{\# \text{coeficientes libres}}{\# \text{evaluaciones}}.$$

La distancia relativa es

$$\delta = \frac{d}{n} = \frac{n - k + 1}{n} = 1 - R + \frac{1}{n} \xrightarrow{n \rightarrow \infty} 1 - R.$$

A.3. Tabla de casos numéricos: RS sobre $\mathbb{F}_q$ con q grande

n	k	grado max. de f	$d = n - k + 1$	$R = k/n$	$\delta = d/n$
8	1	0 (constantes)	8	0,125	1,000
8	2	1	7	0,250	0,875
8	4	3	5	0,500	0,625
8	6	5	3	0,750	0,375
8	8	7	1	1,000	0,125
16	4	3	13	0,250	0,813
16	8	7	9	0,500	0,563
16	12	11	5	0,750	0,313
32	8	7	25	0,250	0,781
32	16	15	17	0,500	0,531
32	24	23	9	0,750	0,281

Lectura de la tabla: subir k (más coeficientes libres en f , más información codificable, más tasa R) implica bajar d (menos protección). El TFA *cuantifica* este intercambio en la fórmula $d = n - k + 1$.

A.4. Una analogía aritmética

La intuición de “inyectividad por TFA” tiene un análogo elegante en aritmética. Sustituyamos el cuerpo $\mathbb{F}$ por $\mathbb{Z}$, los polinomios por enteros, las evaluaciones por restos módulo primos:

Reed–Solomon (algebraico)	Restos chinos (aritmético)
Polinomio $f \in \mathbb{F}[X]$, $\deg f < k$	Entero m , $0 \leq m < \prod_{i=1}^k p_i$
Evaluación $f(\alpha_i)$	Reducción $m \bmod p_i$
n puntos distintos $\alpha_1, \dots, \alpha_n$	n primos distintos $p_1, \dots, p_n$
“ $f_1 - f_2$ no nulo, grado $< k$, $\leq k - 1$ raíces” (TFA)	“ $m_1 - m_2$ no nulo, $ m_1 - m_2 < \prod_{i=1}^k p_i$, comparte $\leq k - 1$ residuos” (CRT)
$d = n - k + 1$	$d \geq n - k + 1$

Esta analogía (que [2] desarrolla en p. 36) muestra que la propiedad “distancia = $n - k + 1$ por inyectividad de evaluaciones” no es exclusiva de la teoría de polinomios: aparece cada vez que tengamos un sistema de “representaciones por evaluación” donde un objeto pequeño quede determinado por pocas evaluaciones.

B. Síntesis y puente a la Clase 2

Lo que sabemos al final de esta clase.

- Un *código* es un subconjunto $\mathcal{C} \subseteq \Sigma^n$. Sus parámetros son (n, k, d) donde $k = \log_q |\mathcal{C}|$ y d es la distancia mínima de Hamming entre palabras-código distintas.
- La *tasa* $R = k/n$ mide eficiencia; la *distancia relativa* $\delta = d/n$ mide robustez.
- Un código de distancia d corrige unívocamente hasta $\lfloor (d-1)/2 \rfloor$ errores y $d-1$ borraduras. Permitiendo listas de tamaño polinomial, se puede ir más allá: la cota de Johnson y la capacidad de list decoding extienden el radio hasta $1 - \sqrt{1-\delta}$ y $1 - R$ respectivamente.
- Si $\Sigma = \mathbb{F}_q$ y $\mathcal{C}$ es subespacio vectorial, decimos que es *lineal* y escribimos $[n, k, d]_q$. La distancia se calcula como mínimo peso de un codeword no nulo.

- Los códigos de *Reed–Solomon* $RS_q[\alpha, k]$ tienen parámetros $[n, k, n - k + 1]_q$. Su construcción se apoya enteramente en una sola consecuencia del TFA: *un polinomio no nulo de grado $< k$ tiene a lo sumo $k - 1$ raíces*.

Lo que viene en la Clase 2.

- Cotas universales sobre (n, k, d) : la *cota de Singleton* ($k \leq n - d + 1$), la cota de Hamming (*sphere-packing*), y la noción de código *MDS* (saturador de Singleton). Verificación de que RS es MDS.
- El problema algorítmico: dado $\mathbf{y} = \mathbf{c} + \mathbf{e}$ con $\text{wt}(\mathbf{e}) \leq \lfloor (n - k)/2 \rfloor$, recuperar $\mathbf{c}$ en tiempo polinomial. **Algoritmo de Berlekamp–Welch**.
- Decodificación por lista efectiva para RS: los algoritmos de Sudan y Guruswami–Sudan, que alcanzan la barrera de Johnson constructivamente.

Referencias

- [1] V. Guruswami, A. Rudra, M. Sudan. *Essential Coding Theory*. Borrador de libro, University at Buffalo, edición 2025. Disponible en <https://cse.buffalo.edu/faculty/atri/courses/coding-theory/book/>.
- [2] V. Guruswami. *List Decoding of Error-Correcting Codes*. PhD Thesis, MIT, 2001 (publicado luego como volumen 3282 de Lecture Notes in Computer Science, Springer).
- [3] V. Guruswami. *Algorithmic Results in List Decoding*. Foundations and Trends in Theoretical Computer Science, vol. 2, no. 2, pp. 107–195, 2006.
- [4] J. H. van Lint, R. M. Wilson. *A Course in Combinatorics*, 2nd ed. Cambridge University Press, 2001 (capítulos 18 y 20).
- [5] M. Kelbert, Y. Suhov. *Information Theory and Coding by Example*. Cambridge University Press, 2013.
- [6] R. C. Singleton. *Maximum distance q -nary codes*. IEEE Trans. Inform. Theory IT-10 (1964), 116–118.
- [7] I. S. Reed, G. Solomon. *Polynomial Codes over Certain Finite Fields*. J. Soc. Indust. Appl. Math. 8 (1960), 300–304.
- [8] C. E. Shannon. *A Mathematical Theory of Communication*. Bell System Technical Journal 27 (1948), 379–423 y 623–656.
- [9] R. W. Hamming. *Error Detecting and Error Correcting Codes*. Bell System Technical Journal 29 (1950), 147–160.
- [10] E. Ben-Sasson, D. Carmon, U. Haböck, S. Kopparty, S. Saraf. *On Proximity Gaps for Reed–Solomon Codes*. Preprint, 6 de noviembre de 2025.
- [11] P. Harsha, M. Kumar, R. Saptharishi, M. Sudan. *An Improved Line-Point Low-Degree Test*. arXiv:2311.12752, 2023.